

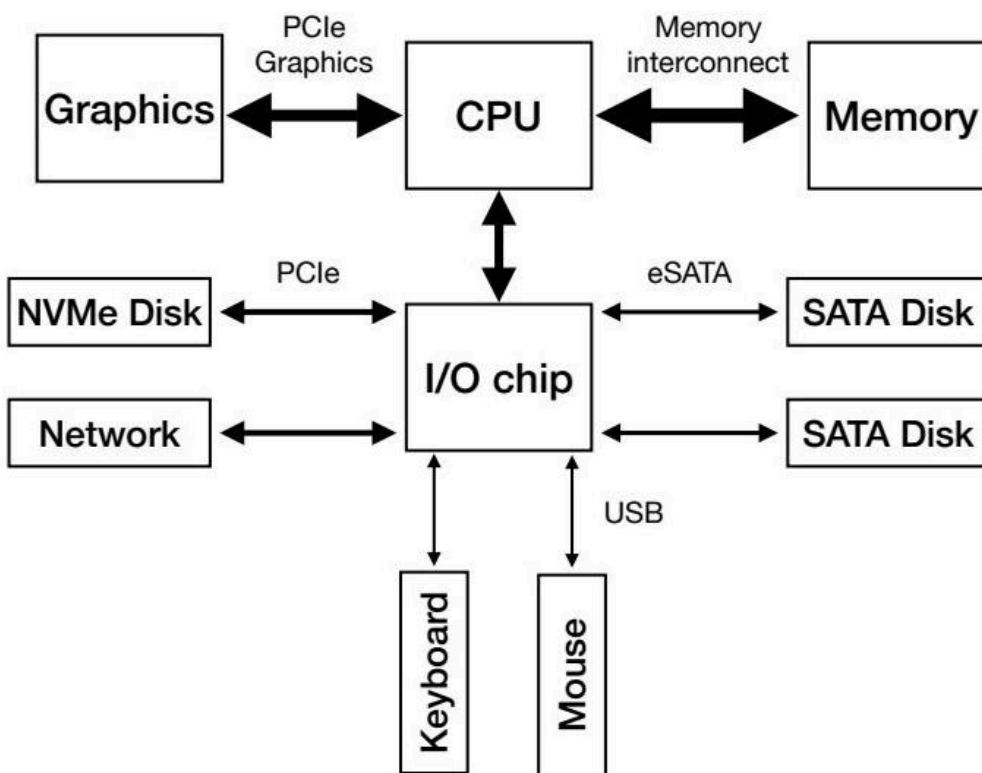
Persistência

Vamos estudar como o sistema operacional (SO) abstrai o acesso aos drivers de dispositivos de baixo nível dos discos, como o SO torna as operações de disco eficientes quando vários programas usam o mesmo dispositivo simultaneamente e como o SO habilita a abstração baseada em arquivos (arquivos, pastas, etc.). Estudaremos as propriedades e o design dos dispositivos de disco para responder a essas perguntas.

Dispositivos de E/S

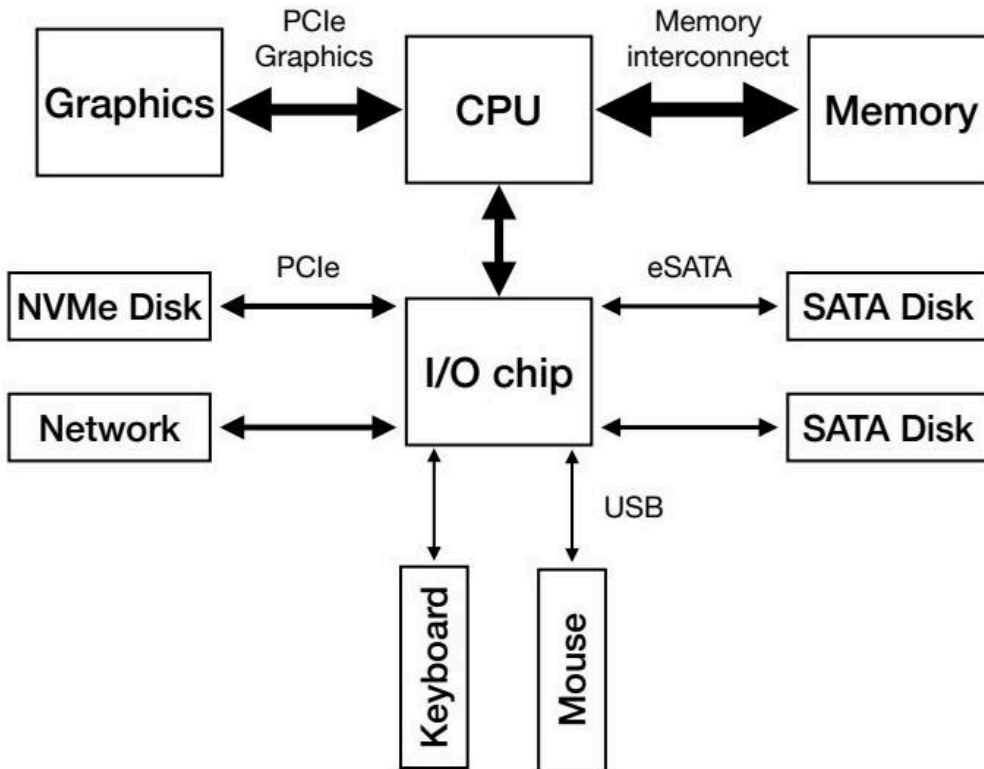
Arquitetura Baseada em Intel

A CPU e a memória são conectadas pelo **BUS** mais rápido. Uma **BUS** de alta velocidade também é usada para placas gráficas.



Modern system architecture (Intel-based)

A CPU se conecta a um chip de E/S através de uma **Direct Media Interface (DMI)**. O **PCIe** é a **BUS** de E/S mais rápida, usada para rede e discos de armazenamento modernos (por exemplo, **NVMe SSDs**). O **eSATA** é usado por discos mais tradicionais (**SATA SSDs, HDDs**). A **BUS USB** é usada para dispositivos de baixo desempenho, como teclados, mouses, discos externos.



Modern system architecture
(Intel-based)

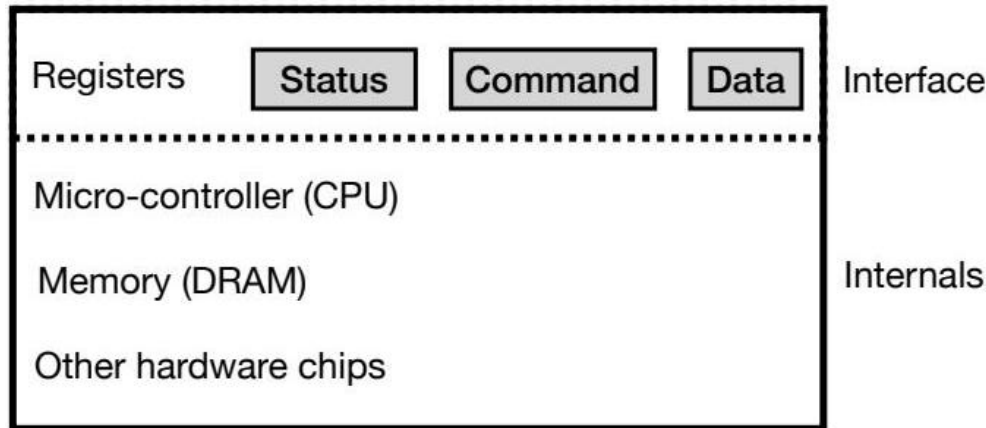
Dispositivo Canônico

Um dispositivo possui duas partes principais:

- A interface apresentada ao sistema.
- Sua estrutura interna, como os chips de hardware (por exemplo, CPU simples, memória) e firmware (o software que é executado no dispositivo).

Em um dispositivo canônico (não um dispositivo real):

- O dispositivo é controlado por meio de três registradores que são lidos e/ou gravados pelo SO:
 - **Status**: Contém o status atual do dispositivo.
 - **Command**: Contém comandos a serem executados pelo dispositivo.
 - **Data**: Contém dados a serem gravados ou lidos do dispositivo.



Dispositivo Canônico

Um protocolo simples para interagir com um dispositivo de disco seria:

1. O SO espera até que o dispositivo esteja pronto, lendo repetidamente o registrador de status (**polling** do dispositivo).
2. O SO grava alguns bytes no registrador de dados. Quando a CPU é responsável por copiar dados para este registrador, nos referimos a esta abordagem como **E/S programada (PIO)**.
3. O SO grava o registrador de comando para informar ao dispositivo que os dados estão presentes e devem ser persistidos.
4. Finalmente, o SO espera que a solicitação termine, pesquisando novamente o dispositivo (registrador de status).

Para o SO interagir com o dispositivo, duas abordagens são possíveis:

- Instruções específicas de E/S para gravar nos registradores são suportadas pelo SO.
- A E/S mapeada na memória é usada, ou seja, os registradores do dispositivo são mapeados como se fossem carregados em regiões de memória, e o SO os usa como memória regular.

Polling vs Interrupções

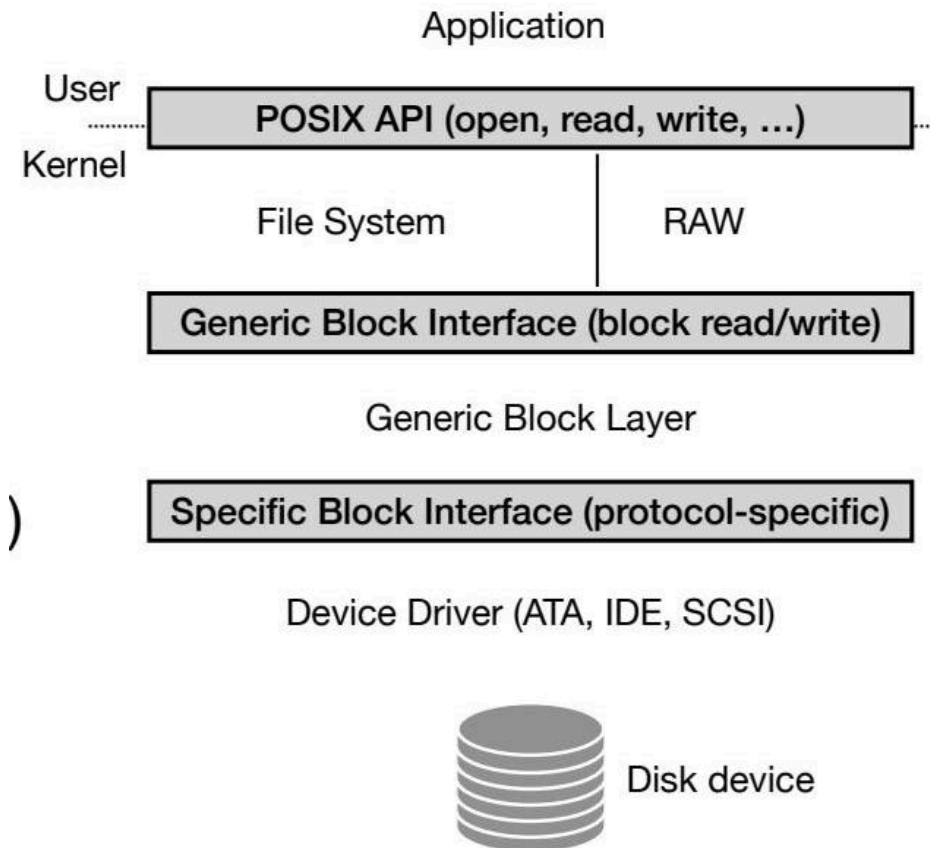
O **polling** parece ineficiente, desperdiça ciclos de CPU esperando que o dispositivo complete sua atividade. O dispositivo gera uma interrupção de hardware quando a solicitação está pronta, enquanto isso, o SO (e a CPU) podem realizar outras tarefas úteis. No entanto, usar interrupções nem sempre pode ser a melhor decisão. Se o dispositivo for muito rápido (ou seja, responde em uma (ou poucas) solicitações de pesquisa), o custo de desempenho da troca de contexto e do tratamento de interrupções pode ser poupado. Quando a velocidade do dispositivo é desconhecida ou varia com o tempo, uma abordagem híbrida é usada. Dispositivos que suportam solicitações paralelas geralmente coalescem (em lote) várias interrupções antes de notificar o SO (ou seja, para evitar o custo de tratá-las independentemente) e a latência (uma compensação típica quando as solicitações são agrupadas).

Acesso Direto à Memória (DMA)

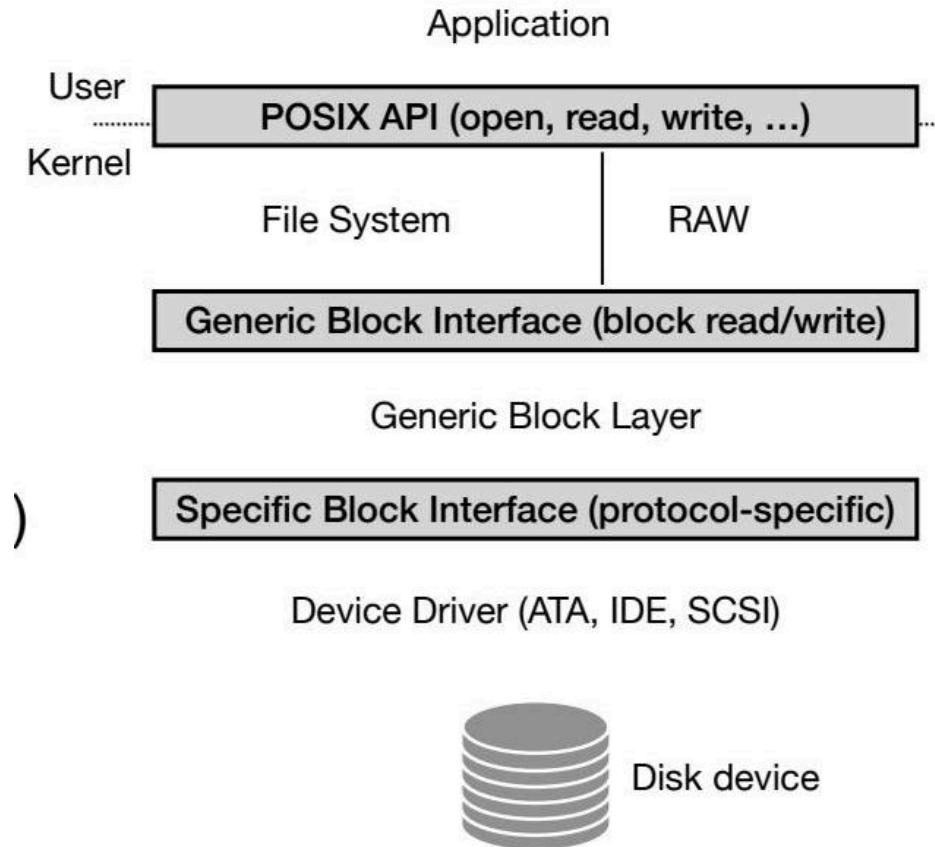
O uso de E/S programada (**PIO**) força a CPU a interromper outras tarefas apenas para copiar dados da memória para o dispositivo de E/S (por exemplo, o registrador do dispositivo). O mecanismo **DMA** é um dispositivo específico que pode orquestrar dados entre os dispositivos de E/S e a memória principal com pouca intervenção da CPU. O SO apenas informa ao(s) dispositivo(s) **DMA** de onde os dados devem ser copiados e, então, a CPU fica livre para realizar outras tarefas.

Driver de Dispositivo

Os dispositivos de E/S suportam diferentes interfaces (por exemplo, [ATA](#), [IDE](#), [SCSI](#)). Como o SO pode abstrair essas interfaces? Solução: a abstração do driver de dispositivo.



Exemplo: Uma pilha de sistema de arquivos Linux simplificada. Os aplicativos acessam um sistema de arquivos ou um dispositivo de bloco [RAW](#), expondo a API [POSIX](#). O sistema de arquivos (e aplicativos que usam um dispositivo de bloco bruto) só veem a interface genérica exposta pela camada de bloco (ou seja, solicitações de leitura e gravação de bloco). A camada de bloco encaminha as solicitações para o driver de dispositivo correto, que sabe como traduzi-las em leituras e gravações específicas do protocolo no dispositivo de E/S.

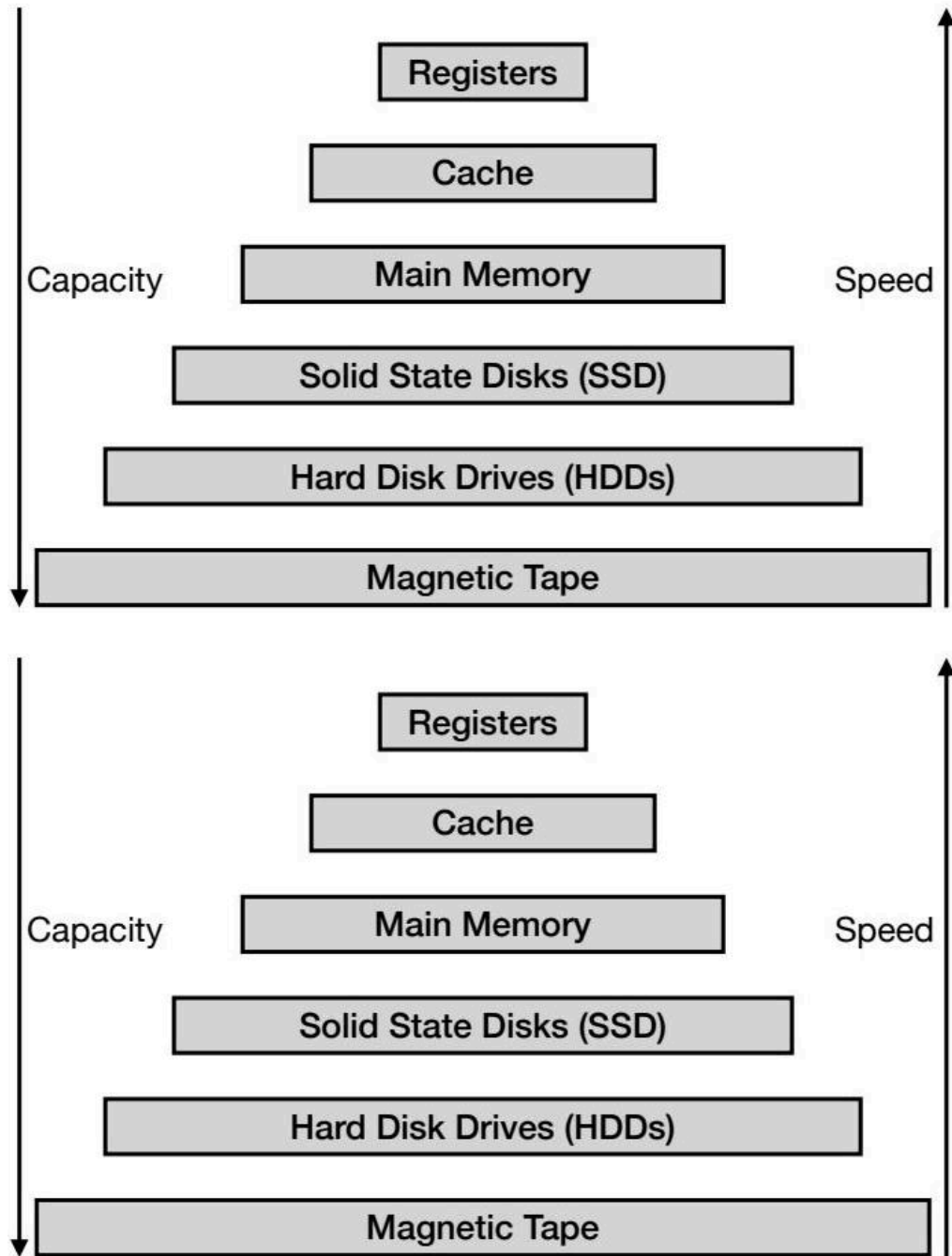


Hierarquia de Memória

Os registradores da CPU, caches e memória principal têm os tempos de acesso mais rápidos (nesta ordem), mas são voláteis.

Trade-offs:

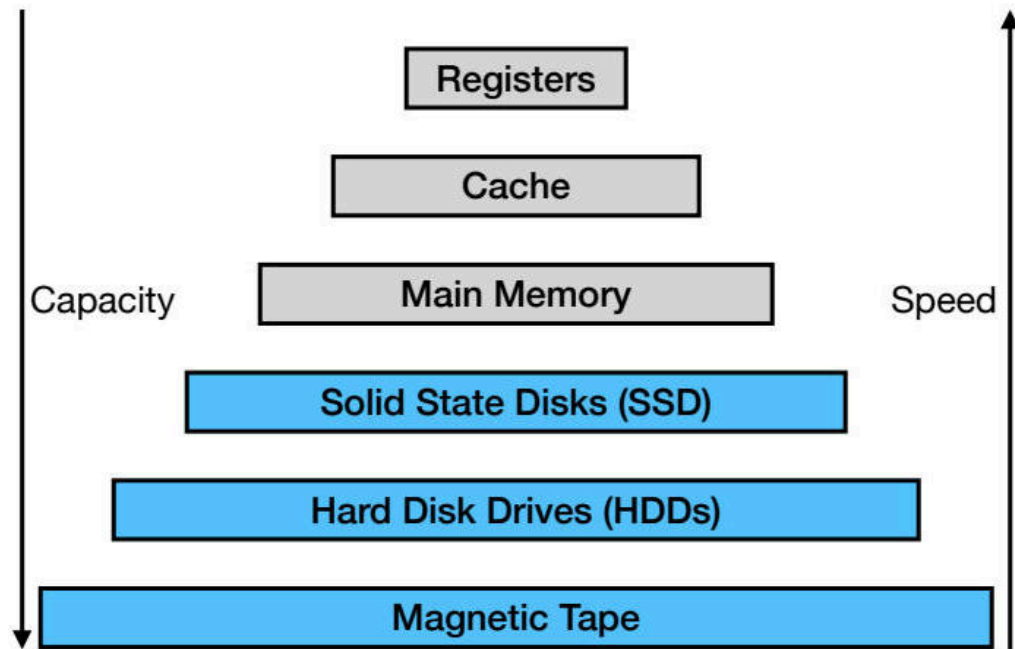
- Dispositivos persistentes, como **SSDs**, **HDDs** e fitas, têm velocidades de acesso mais lentas (nesta ordem).
- À medida que a velocidade aumenta, o custo por byte também é maior.
- Para tornar os sistemas acessíveis (e devido às leis da física), a quantidade de memória rápida disponível é menor do que a quantidade de memória de capacidade mais lenta.



Qual disco devo escolher?

Dada a grande quantidade de dispositivos de armazenamento persistente, qual(is) devemos escolher? Para responder a esta pergunta, devemos:

- Entender os pontos fortes e fracos de cada dispositivo - ou seja, desempenho, custo, capacidade.
- Conhecer a carga de trabalho (ou seja, o tipo de solicitações) que os dispositivos estarão atendendo.



Cargas de Trabalho de Armazenamento

- **Dados de Arquivo:**

- Dados são armazenados para fins de arquivo. Útil para dados que raramente são acessados, mas podem ser relevantes no futuro.

O rendimento é favorecido em detrimento da latência. Grandes quantidades de dados devem ser gravadas/lidas de forma eficiente.

- Cargas de trabalho sequenciais e somente anexação.
 - Os dados de arquivo (por exemplo, arquivos) são gravados e lidos sequencialmente.
 - Os dados de arquivo geralmente são somente anexação (ou seja, dados de gravação única sem atualizações no local).

- **Dados de Backup:**

- Backups de dados novos. Útil para informações que ainda estão em uso e podem ser acessadas frequentemente no futuro.

O rendimento ainda é favorecido em detrimento da latência. Grandes quantidades de dados devem ser gravadas/lidas de forma eficiente.

- Cargas de trabalho principalmente sequenciais (acessos aleatórios também podem acontecer).
 - Às vezes, pode-se querer recuperar/atualizar partes específicas de dados de backup (por exemplo, arquivos específicos ou mesmo pequenas porções destes).
- Os dados agora podem ser atualizados (tipicamente de forma esporádica).
 - Em alguns casos, apenas as diferenças (dados modificados) são armazenadas entre os backups.

- **Armazenamento Primário:**

- Suporte de armazenamento para bancos de dados, análise de dados, estruturas de IA, VMs.

Alto rendimento e/ou baixa latência são agora desejáveis. Grandes quantidades de dados podem ser gravadas/lidas (rendimento). Gravações/leituras de tamanho pequeno devem ser feitas de forma eficiente (latência).

- Cargas de trabalho sequenciais e aleatórias.
 - O conteúdo (por exemplo, de arquivos) pode ser parcialmente acessado e fora de ordem.
- Cargas de trabalho intensivas em dados e metadados.
 - Por exemplo, acesso frequente ao conteúdo (dados) de arquivos, mas também aos seus metadados.
- Espera-se que os dados sejam atualizados com frequência.

Dispositivos de Armazenamento Persistente e as principais cargas de trabalho que eles visam

- **Fita:**
 - Usado para dados de arquivo.
 - Confiável e barato.
 - Suporte limitado para acessos aleatórios e atualizações no local.
- **HDD:**
 - Usado para dados de arquivo, backup e (ainda em alguns casos) primários.
 - Ainda barato, com suporte para acessos aleatórios e atualizações no local.
- **SSD (inclui SATA/NVMe SSDs):**
 - Usado para dados de backup e primários.
 - Mais caro que os HDDs, mas mais rápido, especialmente para acessos aleatórios.
- **Memória Persistente:**
 - Usado para armazenamento primário (principalmente usado como cache).
 - Velocidade mais próxima da **RAM** para cargas de trabalho sequenciais e aleatórias, mas caro.

Estudo de Caso: Discos Rígidos (HDDs)

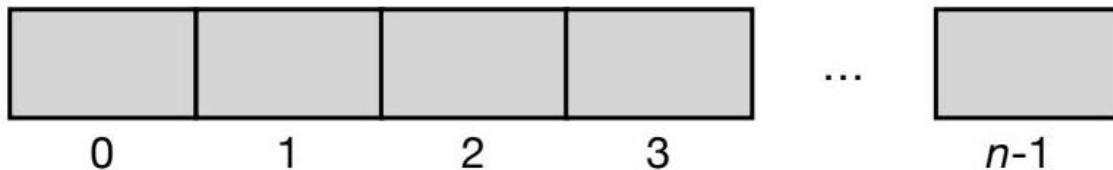
Por que estudar os **HDDs** modernos? Eles têm sido a principal forma de armazenamento de dados persistente por décadas e moldaram o design de sistemas de arquivos amplamente utilizados.

Perguntas:

- Como os **HDDs** armazenam dados e qual é sua interface?
- Como os dados são dispostos e acessados nesses discos?
- Como o agendamento de disco melhora o desempenho?

Interface

- Composto por setores (tipicamente blocos de 512 bytes) que podem ser lidos ou gravados.
 - Para um disco com n setores, estes são numerados de 0 a $n-1$.
 - Um disco pode ser visto como uma matriz de setores (0 a $n-1$ é o espaço de endereço do disco).
- Operações multi-setoriais são possíveis (por exemplo, muitos sistemas de arquivos leem ou gravam 4 KB (ou mais) de cada vez).
 - No entanto, os fabricantes da unidade apenas garantem que uma única gravação de 512 bytes é atômica (ou seja, ela será concluída em sua totalidade ou não será concluída).
 - Se ocorrer uma perda de energia, apenas uma parte de uma gravação maior pode ser concluída (gravação incompleta).



Representação lógica de um disco (uma matriz de setores de 0 a $n-1$)

Geometria de um HDD

- O **platter** é uma superfície dura circular na qual os dados são armazenados.
 - Cada **platter** tem dois lados, cada um dos quais é chamado de superfície.

Mostrador circular com dois círculos concêntricos

- Os dados são codificados em cada superfície em círculos concêntricos de setores chamados **tracks**.
 - Uma única superfície contém milhares e milhares de **tracks**!
- Uma unidade pode ter um ou mais **platters** presos em torno de um **spindle**. O **spindle** é conectado a um motor que gira os **platters** a uma determinada taxa fixa.
 - A taxa para unidades modernas está entre 7200 e 15000 rotações por minuto (**RPM**).
 - Assumindo 10000 **RPM**, uma única rotação leva 6 milissegundos.

Um disco com apenas um único track

- A cabeça do disco é usada para ler ou gravar um setor específico (uma cabeça por superfície).
 - O braço do disco se move pela superfície para posicionar a cabeça na **track** correta.

Um disco com apenas um único track

Geometria de HDDs: Disco de Track Única

Diagrama detalhado de um disco rotativo

Exemplo: Um disco com uma única **track** (setores 0 a 12).

- A cabeça do disco, anexada à extremidade do braço, é colocada no setor 6.
- O **platter** gira no sentido anti-horário.
- Uma solicitação para o setor 0 é feita.
 - Deve-se esperar que o setor 0 esteja sob a cabeça do disco (ou seja, esperar que o **platter** gire).
 - Se uma rotação completa do disco leva R, para este exemplo, deve-se esperar metade de R (atraso rotacional) para que o setor 0 esteja sob a cabeça do disco.

Um disco de track única mais cabeça e braço do disco

Qual solicitação de setor tem o maior atraso rotacional para este exemplo?

Geometria de HDDs: Disco de Múltiplas Tracks

Ilustração detalhada de um disco com tracks concêntricas

Exemplo: Disco com 3 **tracks**, cada uma com 12 setores.

- A cabeça do disco é colocada no setor 30 (**track** mais interna).
- O **platter** gira no sentido anti-horário.
- Uma solicitação para o setor 11 é feita.
 1. O braço deve primeiro se mover para a **track** mais externa (fase de busca).

Ilustração detalhada do prato de um disco rígido

Exemplo: Disco com 3 **tracks**, cada uma com 12 setores.

- A cabeça do disco é colocada no setor 30 (**track** mais interna).
- O **platter** gira no sentido anti-horário.
- Uma solicitação para o setor 11 é feita.

1. O braço deve primeiro se mover para a **track** mais externa (fase de busca).
2. Quando a busca é concluída, o **platter** girou (por exemplo, a cabeça está passando pelo setor 9). Deve-se esperar que o setor 11 esteja sob a cabeça (fase de rotação).

 Ilustração detalhada de um disco com tracks concêntricas

Exemplo: Disco com 3 **tracks**, cada uma com 12 setores.

- A cabeça do disco é colocada no setor 30 (**track** mais interna).
 - O **platter** gira no sentido anti-horário.
 - Uma solicitação para o setor 11 é feita.
1. O braço deve primeiro se mover para a **track** mais externa (fase de busca).
 2. Quando a busca é concluída, o **platter** girou (por exemplo, a cabeça está passando pelo setor 9). Deve-se esperar que o setor 11 esteja sob a cabeça (fase de rotação).
 3. Quando o setor 11 está sob a cabeça, os dados podem ser lidos ou gravados no disco (fase de transferência).

Algumas Considerações

 Diagrama circular representando o layout da track de um disco rígido

- Para garantir que as operações sequenciais sejam devidamente atendidas ao cruzar os limites da **track**, os setores das **tracks** podem ser inclinados.
 - Compensa o tempo de busca, ou seja, quando a cabeça atinge a **track** certa, o setor desejado ainda não girou sob a cabeça.
- As **tracks** externas tendem a ter mais setores do que as internas (mais espaço disponível) e podem ser organizadas em zonas (unidades de disco multi-zonas).
- Os discos geralmente têm um pequeno cache (por exemplo, 8, 16, 32 MB) para armazenar dados lidos ou gravados no disco.
 - Importante: Quando o disco deve reconhecer a conclusão de uma solicitação de gravação? Quando está no cache (write-back) ou quando realmente foi gravado no **platter** do disco (write-through)?

Tempo e Taxa de E/S

O tempo de E/S para ler/gravar um determinado setor ($T_{I/O}$) depende de:

- O tempo para mover o braço do disco para a **track** correta (T_{seek}).
- O tempo para colocar a cabeça do disco sob o setor correto ($T_{rotation}$).
- O tempo para ler/gravar (transferir) o setor ($T_{transfer}$).

$$T_{I/O} = T_{seek} + T_{rotation} + T_{transfer}$$

A taxa de E/S (ou seja, bytes servidos por unidade de tempo - $R_{I/O}$) é calculada dividindo a quantidade de dados a serem transferidos ($TransferSize$) pelo tempo que levou ($T_{I/O}$).

- Por exemplo, se uma transferência de 1MB requer 10 ms de tempo de E/S (0.010 segundos), então nosso disco fornece uma taxa de 100 MB/s para solicitações de 1MB.

$$R_{I/O} = TransferSize / T_{I/O} = 1 / 0.010 = 100MB/s$$

Análise de Tempo e Taxa de E/S Baseada na Carga de Trabalho

- O tempo e as taxas de E/S variam de acordo com as cargas de trabalho que os discos estão atendendo.
 - Cargas de trabalho sequenciais: A leitura de um grande número de setores contíguos por solicitação de E/S (por exemplo, 1MB) requer apenas uma única busca e rotação antes de um longo tempo de transferência.
 - Ou seja, quando a transferência do setor n é concluída, a cabeça do disco está agora sob o setor $n+1$ (assumindo que os setores estão na mesma **track**...).
 - Cargas de trabalho aleatórias: A emissão de pequenas operações de E/S (por exemplo, 4KB) para setores não contíguos exigirá uma busca, rotação e transferência por solicitação.
- Conclusão: Sempre que possível, transfira dados para e de discos sequencialmente. Quando isso não for possível, faça acessos aleatórios com grandes payloads (tamanhos de bloco).

Agendamento de Disco

Devido ao alto custo de E/S de disco, decidir como um conjunto de solicitações de E/S é solicitado ao disco desempenha um papel importante no desempenho. Esta é uma tarefa para o agendador de disco.

Pergunta: Quando o agendador de disco tem várias solicitações de E/S esperando para serem atendidas (em uma fila de E/S), qual(is) deve(m) ser feita(s) primeiro?

Boa notícia! Ao contrário do agendamento de processos, onde a duração de cada trabalho é desconhecida, no agendamento de disco, podemos estimar o tempo de busca e o atraso rotacional de uma determinada solicitação, tendo assim uma ideia do tempo necessário para atendê-la.

Vamos dar uma olhada nas possíveis políticas de agendamento.

Agendamento de Menor Tempo de Busca Primeiro (SSTF)

 Diagrama de uma roda de roleta representando o agendamento de solicitações para setores

Critério de seleção: Atender primeiro as solicitações para a **track** mais próxima (menor tempo de busca).

Exemplo: Cabeça no setor 30 (**track** interna), com solicitações pendentes para os setores 2 e 21.

- Decisão: Primeiro atender o setor 21 (**track** do meio) e depois o 2 (**track** externa).

Problema de implementação: A geometria da unidade (disposição dos setores nas **tracks**) não é conhecida pelo SO (ele apenas vê uma matriz de setores).

- Solução: O SO usa uma política de primeiro bloco mais próximo (**NBF**) para agendar solicitações. Os blocos mais próximos em nossa matriz de setores corresponderão a **tracks** próximas na geometria do disco.

Principal Desvantagem: O **SSTF** sofre de inanição.

- Imagine um fluxo constante de solicitações chegando à **track** interna. As solicitações de outras **tracks** nunca serão atendidas!

Agendamento de Elevador (aka SCAN ou C-SCAN)

SCAN: Mova-se pelo disco da **track** interna para a externa (ou vice-versa), ou seja, faça uma varredura do disco.

 Diagrama de uma roda de roleta representando o agendamento de solicitações para setores

Se uma solicitação chegar para um setor que já foi passado nesta varredura, coloque-o na fila para a próxima varredura (evita a inanição de solicitações distantes). No **SCAN**, as varreduras são feitas em ambas as direções (por exemplo, varredura de **tracks** internas para externas, depois varredura de **tracks** externas para internas e, em seguida, repetição...).

Fato interessante: O **SCAN** também é conhecido como o algoritmo do elevador devido à sua estratégia de varredura. Imagine ir do andar 10 ao 1, alguém entrando no andar 3 e pressionando 4, e o elevador indo para o 4º andar porque está mais perto...

Circular SCAN (C-SCAN): Varre as **tracks** do disco em uma única direção (por exemplo, varredura de **tracks** externas para internas, reinicialização e início novamente nas **tracks** externas).

- O **SCAN** varre as **tracks** para frente e para trás, favorecendo as **tracks** do meio (depois de atender a **track** externa, ele passa pela **track** do meio duas vezes antes de retornar à externa).
- O **C-SCAN**, uma variante do **SCAN**, é um pouco mais justo para as **tracks** internas e externas.

Agendamento de Primeiro Tempo de Posicionamento Mais Curto (SPTF)

 Diagrama de uma roda de roleta representando o agendamento de solicitações para setores

Nossos algoritmos consideram apenas o tempo de busca (ignorando o tempo de rotação).

Exemplo: Cabeça no setor 30, com solicitações pendentes para os setores 16 e 8.

- Qual solicitação deve ser atendida primeiro? Depende...

Se o tempo de busca é alto...

Políticas de Agendamento para HDDs

Em sistemas mais antigos, o agendamento de disco era feito pelo sistema operacional. Nos sistemas modernos, o SO escolhe algumas solicitações e deixa o controlador de disco decidir a melhor maneira de agendá-las (por exemplo, seguindo o SPTF).

Mesmo nos sistemas modernos, o sistema operacional implementa otimizações importantes:

- **Mesclagem de I/O:** Solicitações para setores/blocos contíguos são normalmente mescladas pelo SO antes de serem enviadas para o disco. Isso é importante para enviar menos solicitações ao disco, aproveitar melhor sua largura de banda de I/O e tornar as cargas de trabalho do disco mais sequenciais.
- Em muitos casos, o SO espera (enfileira) algumas solicitações de I/O de programas (ou seja, segue uma abordagem não conservadora de trabalho) antes de emití-las para o disco. Não emitir solicitações imediatamente (ou seja, não seguir uma abordagem conservadora de trabalho) permite otimizações de agendamento e mesclagem.

A parte difícil, como sempre, é decidir quantas solicitações enfileirar antes de enviá-las ao disco.

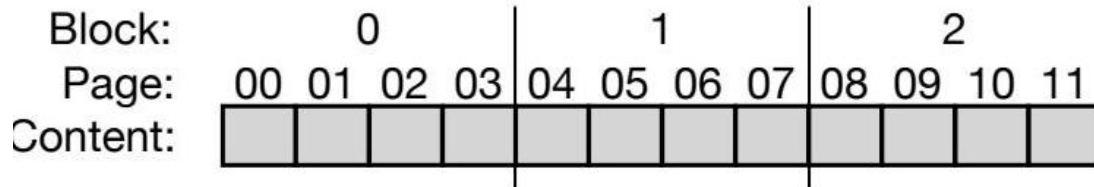
Um bloco pode ser visto como um grupo de um ou mais setores contíguos.

Visão Geral dos Discos de Estado Sólido (SSDs)

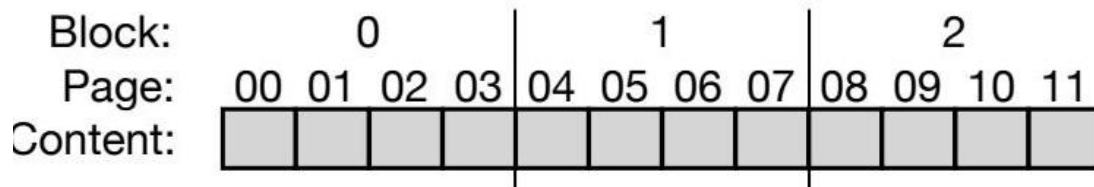
Após décadas de domínio dos HDDs, uma nova forma de armazenamento persistente ganhou força: os Discos de Estado Sólido (SSDs).

- A tecnologia subjacente a estes é conhecida como **flash**, mais especificamente flash baseada em **NAND**.
- SSDs não têm partes móveis mecânicas, como em HDDs. Os bits são armazenados em transistores. Sem tempo de busca do braço ou tempo de rotação da cabeça! Sem erros mecânicos (por exemplo, falhas de cabeça).
- **Blocos** (também chamados de blocos de apagamento), que são tipicamente de tamanho 128KB ou 256KB.
- Dentro de cada bloco, existem múltiplas **páginas**, cada uma com alguns KBs de tamanho (por exemplo, 4KB).

A imagem abaixo ilustra a organização de dados em blocos e páginas dentro de um chip flash.



Um chip flash simples com 3 blocos, cada um com 4 páginas. Não confunda o termo página aqui com páginas de memória. As pessoas de CS tendem a usar o mesmo termo, ou seja, página, bloco, etc., para diferentes contextos.



Características e Desafios Interessantes dos SSDs

- Chips flash permitem a leitura de páginas únicas de forma muito eficiente, independentemente da posição da página no dispositivo e da localização da solicitação anterior. Bom para cargas de trabalho de leitura aleatória (ao contrário dos HDDs)!
- Antes de escrever/atualizar uma página, o bloco completo contendo essa página deve ser apagado. Operação cara em termos de desempenho que apaga o conteúdo de todas as páginas no bloco (ou seja, o conteúdo de páginas válidas deve ser copiado para outro lugar antes de apagar seu bloco).
- Uma vez que um bloco foi apagado, as páginas correspondentes podem ser programadas (ou seja, escritas com novo conteúdo). Mais eficiente (em termos de tempo) do que apagar um bloco, mas menos eficiente do que ler uma página.

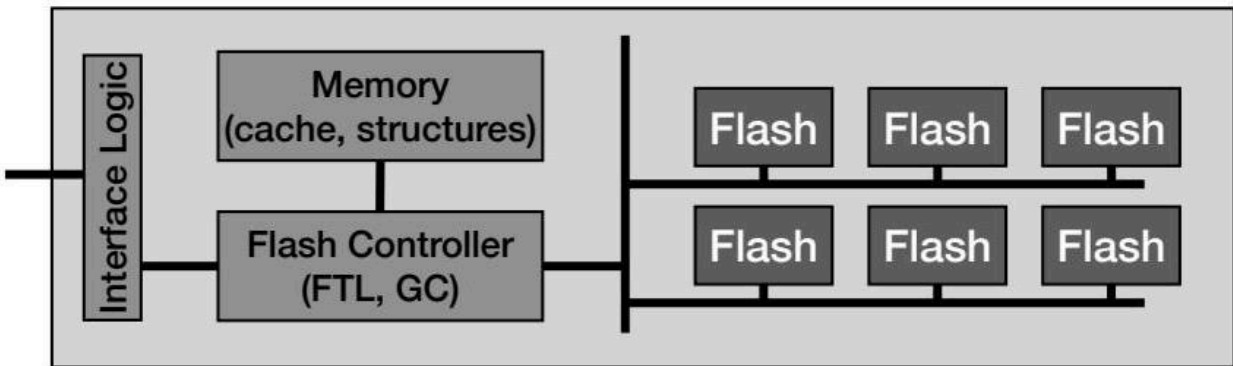
Desgaste e Distúrbio em SSDs ☒

- Apagar um bloco também desgasta a vida útil do drive SSD. Cada bloco só pode ser apagado e programado uma certa quantidade de vezes (por exemplo, 10000 vezes). Este desafio motivou vários algoritmos de nivelamento de desgaste que espalham as operações de apagar/programar (ou seja, carga de escrita) por todos os blocos do dispositivo de forma uniforme para estender sua vida útil.
- Às vezes, ao programar (escrever em) uma página dentro de um chip flash, páginas vizinhas podem ter alguns bits invertidos, um fenômeno conhecido como distúrbio.

Abstração de Disco em SSDs

- Como múltiplos chips flash são fornecidos como um dispositivo de armazenamento tradicional? O dispositivo SSD oferece uma interface de bloco (setor) padrão sobre os chips flash.
- A **Flash Translation Layer (FTL)** mapeia endereços lógicos (interface do dispositivo) para os blocos e páginas físicas correspondentes (chips flash).
- O **Garbage Collector (GC)** rastreia páginas não utilizadas e apaga blocos.
- Estruturas de dados para a FTL e GC, juntamente com caches de dados, são mantidas na memória volátil (SRAM) no dispositivo.

A imagem abaixo representa um diagrama lógico de um SSD baseado em flash, mostrando a interação entre a interface lógica, memória cache, controlador flash (FTL, GC) e os chips flash.



Resumo: HDDs vs SSDs

Quando comparados aos HDDs, os SSDs:

- Se destacam no desempenho de escrita aleatória e leitura aleatória.
- Oferecem melhor desempenho para cargas de trabalho sequenciais (embora a diferença seja menor).
- Lembre-se de que os ganhos de desempenho normalmente aumentam os custos monetários.
- Embora em SSDs, a lacuna entre o desempenho de cargas de trabalho sequenciais e aleatórias seja menor, ainda vale a pena (especialmente para aplicações exigentes de I/O) reduzir as operações de I/O aleatórias.

Integridade e Proteção de Dados: Modos de Falha de Disco

Discos podem falhar. Quando isso acontece, pode levar à indisponibilidade de dados e até mesmo à perda de dados. Vamos rapidamente repassar os modos de falha de disco comuns:

- **Modelo de falha-parada (Fail-stop):** Ou o disco está funcionando ou falha completamente.
 - **Modelo de falha parcial (Fail-partial):** O disco falha parcialmente, ou seja, alguns setores não estão acessíveis.
- Dois tipos de falhas de bloco único são comuns e merecem consideração:

- **Erros de setor latentes (LSEs):** Um setor (ou um grupo deles) é danificado (por exemplo, falha de cabeça de disco, raios cósmicos).
 - Códigos de correção de erros (ECC) no disco podem ser usados para determinar se os setores estão danificados e, em alguns casos, para corrigi-los.
 - Os discos têm setores sobressalentes que são usados para substituir setores defeituosos. Feito pelo controlador de disco, de forma transparente para o SO!
- **Corrupção:** Em alguns casos, os setores (blocos) podem se corromper de uma forma não detectável pelo disco.
 - Falhas silenciosas são complicadas, pois o disco pode retornar dados corrompidos para o programa.
 - Checksums (pequeno resumo de um bloco), tipicamente armazenados junto com o conteúdo do bloco, podem ser usados para detectar corrupção de dados (ou seja, o checksum dos dados armazenados anteriormente deve corresponder ao do conteúdo que está sendo lido).
- **Escrita mal direcionada (Misdirected write):** O conteúdo é escrito no deslocamento de disco errado (por exemplo, devido a um bug no controlador de disco).
 - Pode-se adicionar a localização física do bloco ao checksum.
- **Escritas perdidas (Lost writes):** O disco diz ao SO que uma escrita foi feita, mas, de fato, o conteúdo nunca foi persistido.
 - Pode-se ler cada bloco após escrevê-lo para ter certeza, mas isso tem uma penalidade de alto desempenho.
 - Alguns sistemas armazenam o checksum em uma localização de disco diferente (ou seja, em vez de armazená-lo junto com o setor/bloco). A probabilidade de ter uma confirmação ruim do disco para duas escritas independentes é menor do que para uma única...

Checksumming é um recurso importante para a resiliência de dados. De fato, sistemas críticos normalmente limpam seus discos, validando todos os checksums do conteúdo armazenado para detectar, em um estágio inicial, potencial corrupção de dados.

No entanto, checksumming introduz sobrecarga de espaço (espaço adicional em disco para armazenar checksums) e sobrecarga de processamento (checksums devem ser calculados quando os dados são escritos/lidos).

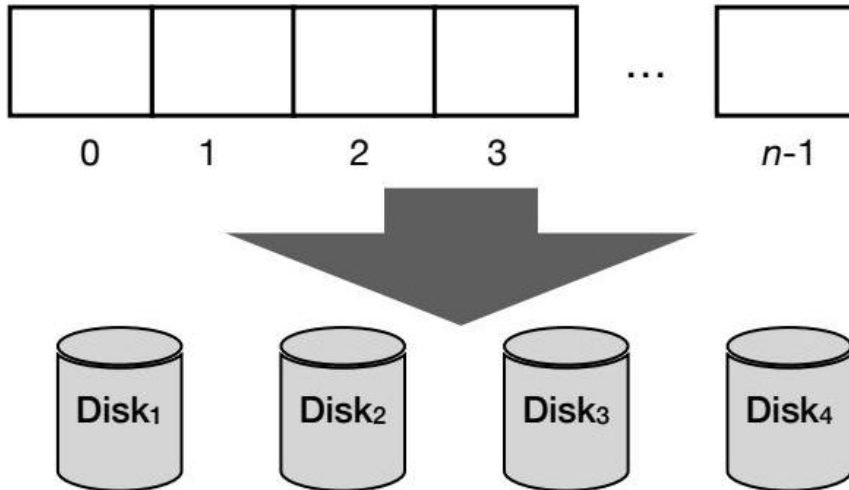
Finalmente, quando um disco falha total ou parcialmente, o que podemos fazer para recuperar nossos dados? Precisamos de redundância!

Técnica e Abstração: Redundant Array of Inexpensive Disks (RAID)

RAID: Técnica que usa múltiplos discos em conjunto para construir um sistema de disco mais rápido, maior e mais confiável.

- Externamente, um sistema RAID parece um único disco, ou seja, um grupo de blocos que se pode ler e escrever - transparência!
- Internamente, é um sistema complexo que requer múltiplos discos e um controlador (com memória volátil e um processador). RAID pode ser implementado através de hardware ou software.

A imagem abaixo ilustra a representação lógica e física de um sistema de disco RAID, mostrando como um array de blocos é distribuído entre múltiplos discos.



RAID Nível 0: Striping

- **RAID nível 0:** Blocos (por exemplo, 4KB) são espalhados por vários discos em um esquema de round-robin.
 - Blocos na mesma linha formam uma *stripe* (em nosso exemplo, blocos 0, 1, 2 e 3 estão na mesma stripe).

Vantagens:

- Mais capacidade (aumento de 4x no espaço de armazenamento em nosso exemplo).
- Paralelismo aprimorado (desempenho) ao acessar chunks contíguos (ou seja, uma stripe).

Desafios / desvantagens:

- Blocos menores levam a um melhor paralelismo, mas pequenos acessos a disco são caros. Encontrar o tamanho de bloco certo para striping nem sempre é fácil.
- RAID 0 não oferece confiabilidade. Se um disco falhar, os dados serão perdidos.

Disco 0	Disco 1	Disco 2	Disco 3
0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

stripe

RAID Nível 1: Mirroring зеркало

- **RAID nível 1:** Espelha os blocos entre múltiplos discos (pode ser combinado com RAID nível 0).

Vantagens:

- Resiliência, ou seja, se um disco falhar, seus blocos podem ser recuperados do disco espelho.
- Os dados podem ser lidos de múltiplos discos (possibilidade de ter paralelismo de leitura e otimizações de balanceamento de carga).

Desafios/desvantagens:

- Em termos de capacidade, não se aproveita ter mais discos (Em nosso exemplo, só temos a capacidade de um disco para armazenar dados, mas precisamos de dois para redundância).
- As escritas devem ser feitas em todos os discos espelhados. Se feito em paralelo, pode levar tanto tempo quanto o disco mais lento.

Qual é a capacidade extra necessária pelo RAID 1, mantendo a resiliência?

Disco 0	Disco 1
0	0
1	1
2	2
3	3

mirror

Disco 0	Disco 1	Disco 2	Disco 3	Disco 4	Disco 5	Disco 6	Disco 7
0	0	1	1	2	2	3	3
4	4	5	5	6	6	7	7
8	8	9	9	10	10	11	11
12	12	13	13	14	14	15	15

RAID 1 sobre dois discos RAID 1+0 (stripe de espelhos)

RAID Nível 4: Bloco de Paridade

- **RAID nível 4:** Adiciona um disco de paridade. Este disco contém um bloco de paridade (calculado com XOR) para cada stripe de blocos.

Vantagens:

- Tolerar 1 falha de disco, ou seja, se um dos discos falhar, o bloco de paridade pode ser usado para reconstruir a stripe.
- O paralelismo de solicitação de leitura é possível em todos os discos (exceto o de paridade).
- Requer apenas 1 disco adicional (menor sobrecarga de espaço de armazenamento do que RAID 1+0).

Desafios / Desvantagens:

- Toda vez que um bloco é atualizado, o bloco de paridade para a stripe deve ser recalculado (o que pode incluir a leitura do conteúdo de outros blocos na stripe). Assim, o disco de paridade é um gargalo de desempenho!

Disco 0	Disco 1	Disco 2	Disco 3	Disco 4
0	1	2	3	P0
4	5	6	7	P1
8	9	10	11	P2
12	13	14	15	P3

Blocos de paridade

RAID Nível 5: Paridade Rotativa

- **RAID nível 5:** Espalha os blocos de paridade pelos discos RAID.

Vantagens:

- Como no RAID 4, tolera 1 falha de disco com um único disco extra.
- Em termos de desempenho, as solicitações de leitura podem aproveitar todos os discos (já que os blocos de paridade são espalhados pelos discos).
- Espalhar os blocos de paridade também alivia o gargalo de recálculo de paridade.

Desafios / desvantagens:

- Não elimina completamente a sobrecarga do recálculo de paridade (mas evita reescrever os blocos de paridade sempre no mesmo disco).
- Mais difícil de implementar e tolera apenas uma única falha de disco. Existem outras variantes de RAID (por exemplo, RAID nível 6) para tolerar múltiplas falhas de disco.

Disco 0	Disco 1	Disco 2	Disco 3	Disco 4
0	1	2	3	P0
5	6	7	P1	4
10	11	P2	8	9
15	P3	12	13	14
P4	16	17	18	19

Blocos de paridade

Observe que os blocos de paridade ainda não podem ser usados para atender às solicitações de leitura. Mas como agora os blocos são espalhados pelos 5 discos, a largura de banda de leitura agregada aumenta.